

```

import json
import math

class VAPScoringEngine:
    """
    Reference Implementation of the Vibe Audio Protocol (V.A.P.) v3.1
    Logic Architecture.
    This engine converts raw audio analysis data into standardized
    V.A.P. Metadata.
    """

    def __init__(self):
        self.version = "3.1"

    # --- PHASE I: PHYSICAL ANALYSIS (DSP Layer) ---

    def calculate_kick_profile(self, attack_ms):
        """Logic for Subset 1.3: Percussive DNA"""
        if attack_ms < 10:
            return "Sharp (Click)"
        elif 10 <= attack_ms <= 30:
            return "Punch (Thud)"
        else:
            return "Boom (Sub)"

    def calculate_spectral_color(self, centroid_hz):
        """Logic for Subset 3.1: Spectral Physics -> Tonal Color"""
        if centroid_hz < 200:
            return "Dark/Muddy"
        elif 200 <= centroid_hz <= 2000:
            return "Warm/Body"
        else:
            return "Bright/Airy"

    # --- PHASE II: PSYCHOLOGICAL ANALYSIS (NLP/ML Layer) ---

    def calculate_thayer_coordinates(self, key_mode, sentiment_score,
rms_amplitude):
        """
        Logic for Subset 5.1: Affective (Thayer Model)
        key_mode: 'Major' or 'Minor'
        sentiment_score: -1.0 to 1.0 (from NLP)
        rms_amplitude: 0.0 to 1.0 (Loudness)
        """
        # Calculate Valence (Positivity)
        base_valence = 0.5 if key_mode == 'Major' else -0.5
        valence = (base_valence + sentiment_score) / 2
        valence = max(-1.0, min(1.0, valence)) # Clamp values

```

```

# Calculate Arousal (Energy)
arousal = rms_amplitude

return {
    "valence": round(valence, 2),
    "arousal": round(arousal, 2),
    "mood_quadrant": self._get_quadrant(valence, arousal)
}

def _get_quadrant(self, valence, arousal):
    if valence > 0 and arousal > 0.5: return "Euphoria/Joy"
    if valence > 0 and arousal <= 0.5: return "Calm/Content"
    if valence <= 0 and arousal > 0.5: return "Anger/Fear"
    return "Depression/Melancholy"

# --- PHASE III: ENVIRONMENTAL ANALYSIS (I/O Layer) ---

def calculate_photometric_hex(self, dominant_freq_hz):
    """
    Logic for Subset 7.1: Photometric (Chromatic Map)
    Maps Audio Frequency to Visual Wavelength (Approximation)
    """
    if dominant_freq_hz < 60:
        return "#8B0000" # Deep Red (Sub Bass)
    elif 60 <= dominant_freq_hz < 250:
        return "#FF8C00" # Dark Orange (Low Mids)
    elif 250 <= dominant_freq_hz < 2000:
        return "#008080" # Teal (Mids/Vocals)
    else:
        return "#4B0082" # Indigo/UV (Highs/Air)

def calculate_kinetic_met(self, bpm):
    """Logic for Subset 8.1: Kinetic (Biometrics)"""
    if bpm < 60:
        return 1.0 # Rest
    elif 60 <= bpm < 100:
        return 3.0 # Light Activity
    elif 100 <= bpm < 140:
        return 6.0 # Moderate Activity
    else:
        return 8.0 # High Intensity (Sprint)

# --- MASTER GENERATOR ---

def generate_vap_profile(self, raw_data):
    """
    Orchestrates the analysis phases to build the full V.A.P.

```

```

Object.
    """

    # Phase I
    kick_profile =
self.calculate_kick_profile(raw_data['attack_ms'])
    spectral_tone =
self.calculate_spectral_color(raw_data['centroid_hz'])

    # Phase II
    affective = self.calculate_thayer_coordinates(
        raw_data['key_mode'],
        raw_data['sentiment_score'],
        raw_data['rms_amplitude']
    )

    # Phase III
    chroma_hex =
self.calculate_photometric_hex(raw_data['dominant_freq_hz'])
    met_score = self.calculate_kinetic_met(raw_data['bpm'])

    # Construct JSON
    vap_object = {
        "VAP_VERSION": self.version,
        "PILLARS": {
            "STRUCTURAL": {
                "PERCUSSIVE_DNA": kick_profile
            },
            "TIMBRAL": {
                "SPECTRAL_PHYSICS": spectral_tone
            },
            "AFFECTIVE": affective,
            "PHOTOMETRIC": {
                "PRIMARY_HEX": chroma_hex
            },
            "KINETIC": {
                "MET_SCORE": met_score,
                "TARGET_HR_ZONE": f"{int(raw_data['bpm'] -
20)}-{int(raw_data['bpm'] + 10)}"
            }
        }
    }
    return json.dumps(vap_object, indent=2)

# --- SIMULATION ---

# Mock Data (representing "After Dark" by Mr.Kitty)
mock_audio_analysis = {

```

```
"bpm": 135,
"attack_ms": 45,          # Slow attack (Synth wash)
"centroid_hz": 2200,      # Bright/Airy
"key_mode": "Minor",
"sentiment_score": -0.2,  # Melancholic
"rms_amplitude": 0.75,    # Steady loud
"dominant_freq_hz": 3000  # High synth lead focus
}

# Run Engine
engine = VAPScoringEngine()
vap_profile = engine.generate_vap_profile(mock_audio_analysis)

print("--- V.A.P. v3.1 Generated Metadata ---")
print(vap_profile)
```